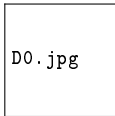


Lecture 21: SQL DML (SELECT Basics)

FROM, WHERE, ORDER BY, TOP, NULL, and Debugging Queries

Z2004: Database Management Systems



D0.jpg

IIT Madras Zanzibar

Tuesday, 21 April 2026

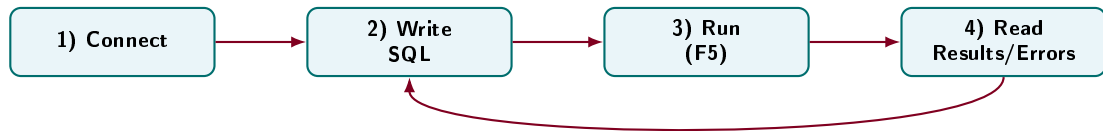
- 1 Live setup: create dataset in SSMS
- 2 SELECT anatomy + choosing columns + aliases
- 3 WHERE filtering (operators, NULL, LIKE, precedence)
- 4 ORDER BY + TOP
- 5 Live coding quiz + mini-assessment

After this lecture, you can:

- ➊ Write correct `SELECT ... FROM ...` queries (no `SELECT *` by default).
- ➋ Filter rows using `WHERE` with `AND/OR/NOT`, `BETWEEN`, `IN`, `LIKE`, and `IS NULL`.
- ➌ Sort results using `ORDER BY` and limit output using `TOP` (SQL Server).
- ➍ Predict common mistakes (`NULL`, precedence, quotes, wrong database) and debug using SSMS messages.
- ➎ Solve practical query tasks on your own machine (live assessment).

Tooling: Microsoft SQL Server + **SQL Server Management Studio (SSMS)**.

SSMS live workflow



Debug rule: fix the **first** error message, then run again.

Live setup (Part 1A): safety + database + schema

Run this first.

```
-- 0) Safety
SELECT DB_NAME() AS CurrentDB;

-- 1) Create database if needed
IF DB_ID('SchoolDB') IS NULL
    CREATE DATABASE SchoolDB;

-- 1b) Switch into it (run this after the CREATE finishes)
USE SchoolDB;

-- 2) Create schema if needed
IF SCHEMA_ID('school') IS NULL
    EXEC('CREATE SCHEMA school');
```

Live setup (Part 1B): drop old tables + create Student

Run this next.

```
-- 3) Drop tables if they exist (order matters: child first)
DROP TABLE IF EXISTS school.Enrolment;
DROP TABLE IF EXISTS school.Course;
DROP TABLE IF EXISTS school.Student;

-- 4A) Create Student
CREATE TABLE school.Student(
  SID    INT IDENTITY(1,1) NOT NULL,
  Name   NVARCHAR(100)     NOT NULL,
  Dept   CHAR(5)           NOT NULL,
  GPA    DECIMAL(3,2)      NULL,
  CONSTRAINT PK_Student PRIMARY KEY (SID),
  CONSTRAINT CHK_Student_GPA CHECK (GPA IS NULL OR GPA BETWEEN 0 AND 4)
);
```

Live setup (Part 1C): create Course

Run this after Student.

```
CREATE TABLE school.Course(  
  CID      CHAR(5)      NOT NULL,  
  CName    NVARCHAR(200) NOT NULL,  
  Credits  SMALLINT     NOT NULL,  
  CONSTRAINT PK_Course PRIMARY KEY (CID)  
);
```

Live setup (Part 1D): create Enrolment (FKs)

Run this last (it references Student and Course).

```
CREATE TABLE school.Enrolment(  
  SID      INT      NOT NULL,  
  CID      CHAR(5)  NOT NULL,  
  Grade    CHAR(2)  NULL,  
  CONSTRAINT PK_Enrolment PRIMARY KEY (SID, CID),  
  CONSTRAINT FK_Enrolment_Student FOREIGN KEY (SID)  
    REFERENCES school.Student(SID),  
  CONSTRAINT FK_Enrolment_Course FOREIGN KEY (CID)  
    REFERENCES school.Course(CID)  
);
```


What did we just create? (tables + keys)

school.Student

PK: SID

Name

Dept

GPA

CHECK: GPA BETWEEN 0 AND 4 (or NULL)

school.Course

PK: CID

CName

Credits

school.Enrolment

PK: (SID, CID)

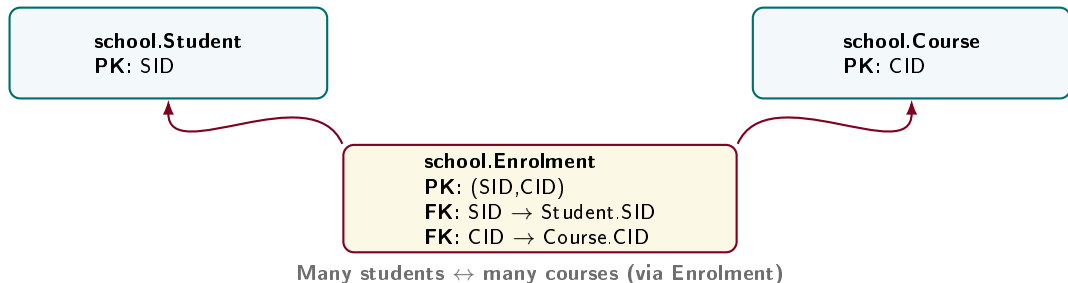
FK: SID → school.Student(SID)

FK: CID → school.Course(CID)

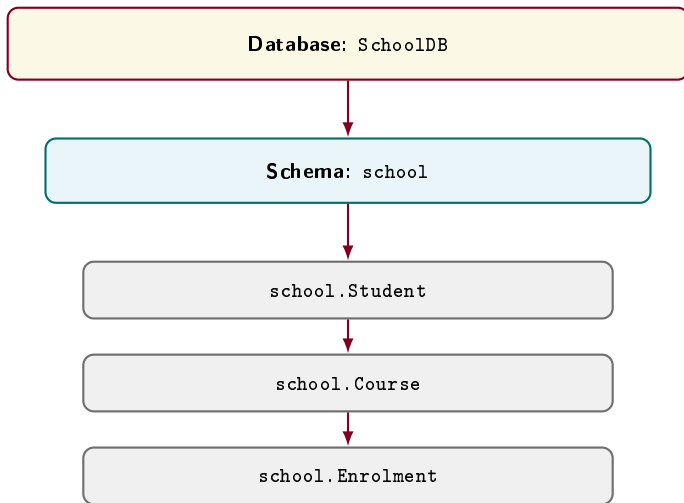
Grade

Idea: Enrolment is a **junction table** for a many-to-many relationship.

How the tables connect (FK arrows)



Where do these objects live? (database $\rightarrow$ schema $\rightarrow$ tables)



Live setup (Part 2A): insert rows into Student + Course

```
USE SchoolDB;
```

```
INSERT INTO school.Student(Name, Dept, GPA) VALUES
```

```
(N'Asha Ali', 'CSE ', 3.80),
```

```
(N'Musa Omar', 'CSE ', 3.10),
```

```
(N'Salma Noor', 'EEE ', NULL),
```

```
(N'John M.', 'CIV ', 2.40),
```

```
(N'Mary K.', 'EEE ', 3.90);
```

```
INSERT INTO school.Course(CID, CName, Credits) VALUES
```

```
('DB101', N'Intro to Databases', 3),
```

```
('PR101', N'Programming I', 4),
```

```
('NW101', N'Networks I', 3);
```

Live setup (Part 2B): insert Enrolment + sanity check

```
USE SchoolDB;

INSERT INTO school.Enrolment(SID, CID, Grade) VALUES
  (1, 'DB101', 'A'), (1, 'PR101', 'B'),
  (2, 'DB101', 'B'),
  (3, 'NW101', NULL),
  (5, 'DB101', 'A');

SELECT * FROM school.Student;
SELECT * FROM school.Enrolment;
```

Sanity check: you should see 5 students and 5 enrolments.

Sample data snapshot (what is in the tables?)

school.Student

SID	Name	Dept	GPA
1	Asha Ali	CSE	3.80
3	Salma Noor	EEE	NULL
5	Mary K.	EEE	3.90

school.Course

CID	CName	Credits
DB101	Intro to Databases	3
PR101	Programming I	4
NW101	Networks I	3

school.Enrolment

SID	CID	Grade
1	DB101	A
1	PR101	B
3	NW101	NULL
5	DB101	A

Notice: Enrolment rows must match existing **SID** and **CID**.

Constraints in action (try to break the rules)

Run these one-by-one. Each should **fail**.

```
-- 1) CHECK constraint (GPA out of range)
INSERT INTO school.Student(Name, Dept, GPA)
VALUES (N'Test BadGPA', 'CSE ', 6.00);

-- 2) PRIMARY KEY uniqueness (duplicate CID)
INSERT INTO school.Course(CID, CName, Credits)
VALUES ('DB101', N'Duplicate Code', 3);

-- 3) FOREIGN KEY (SID does not exist)
INSERT INTO school.Enrolment(SID, CID, Grade)
VALUES (999, 'DB101', 'A');
```

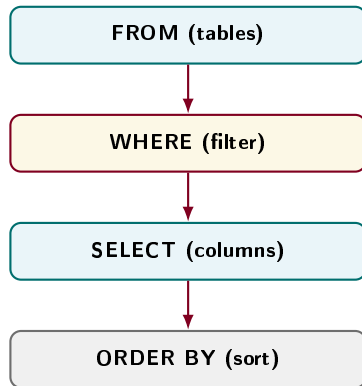
Point: constraints stop bad data *before it enters the table*.

What does SELECT do?

A SELECT query is a pipeline:

- ▶ Pick a table (FROM)
- ▶ Filter rows (WHERE)
- ▶ Choose output columns / expressions (SELECT)
- ▶ Sort (ORDER BY)
- ▶ Limit rows (TOP)

Important: the **logical** order is not the same as the **written** order.



Two orders to remember: written vs logical

Written order (what you type)

► SELECT FROM WHERE ORDER BY

Logical order (what SQL does first)

► FROM → WHERE → SELECT → ORDER BY

Memory line: F-W-S-O = *From, Where, Select, Order*.

Written: SELECT FROM WHERE ORDER BY



Logical: FROM → WHERE → SELECT → ORDER BY

SELECT: minimum working query

```
-- 1) Always confirm DB (avoid the wrong database!)
SELECT DB_NAME() AS CurrentDB;

-- 2) Minimum query
SELECT Name
FROM    school.Student;
```

In this lecture we always write `schema.Table` (e.g., `school.Student`) so queries are unambiguous.

Selecting columns: avoid SELECT * as a habit

```
-- Exploration (OK)
SELECT *
FROM school.Student;

-- Better (explicit)
SELECT SID, Name, Dept, GPA
FROM school.Student;
```

Why avoid SELECT * in real projects?

- ▶ Schema changes can break code unexpectedly
- ▶ Transfers unnecessary columns (slower)
- ▶ Makes reviews harder (what are we using?)

Aliases and computed columns

```
-- Output column names (aliases)
SELECT Name AS StudentName,
       Dept AS Department,
       GPA AS GradePoint
FROM school.Student;

-- Computed columns
SELECT Name,
       GPA,
       GPA * 25 AS ScoreOutOf100
FROM school.Student;
```

Note: aliases are for output; you generally cannot use them in WHERE (because WHERE is evaluated earlier).

DISTINCT: duplicates removal

```
-- Distinct departments
SELECT DISTINCT Dept
FROM school.Student;

-- Distinct (Dept, GPA) pairs
SELECT DISTINCT Dept, GPA
FROM school.Student;
```

Key idea: DISTINCT applies to the **entire selected row** (all columns in the SELECT list).

Common mistake: students expect distinct on only the first column.

WHERE: comparison + range + membership

-- Comparison

```
SELECT SID, Name, GPA
FROM school.Student
WHERE GPA >= 3.50;
```

-- BETWEEN is inclusive (>= and <=)

```
SELECT Name, GPA
FROM school.Student
WHERE GPA BETWEEN 3.00 AND 3.80;
```

-- IN (set membership)

```
SELECT Name, Dept
FROM school.Student
WHERE Dept IN ('CSE', 'EEE');
```

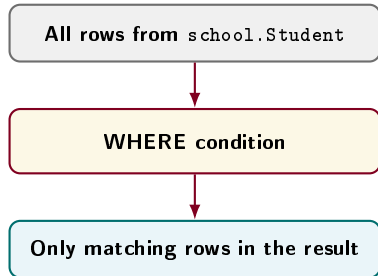
WHERE is a filter (visual intuition)

Think of WHERE as a **sieve**:

- ▶ every row is tested
- ▶ rows that make the condition TRUE pass through
- ▶ FALSE or UNKNOWN (NULL) are removed

Example condition:

- ▶ `GPA >= 3.50`



WHERE: LIKE patterns (text search)

```
-- % = any number of characters
SELECT Name
FROM school.Student
WHERE Name LIKE 'M%';

-- _ = exactly one character
SELECT Name
FROM school.Student
WHERE Name LIKE '_ohn%';
```

Quick pattern cheat sheet

- ▶ 'A%' starts with A
- ▶ '%a' ends with a
- ▶ '%ali%' contains ali
- ▶ '%_a%' 2nd letter is a

NULL in WHERE: the rule you must memorize

```
-- WRONG (returns no rows)
```

```
SELECT Name
FROM school.Student
WHERE GPA = NULL;
```

```
-- Correct
```

```
SELECT Name
FROM school.Student
WHERE GPA IS NULL;
```

```
SELECT Name
FROM school.Student
WHERE GPA IS NOT NULL;
```

Why? NULL means *unknown*, so NULL = NULL is not true.

Exam-style line: use IS NULL / IS NOT NULL.

Why NULL breaks logic (3-valued logic picture)

In SQL, comparisons with NULL produce **UNKNOWN**.

Expression	Result	Kept by WHERE?
GPA = 3.5 when GPA=3.5	TRUE	Yes
GPA = 3.5 when GPA=2.4	FALSE	No
GPA = 3.5 when GPA=NULL	UNKNOWN	No

Key rule: WHERE keeps only rows where the condition is **TRUE**.

TRUE $\Rightarrow$ row stays

FALSE $\Rightarrow$ row removed

UNKNOWN (NULL) $\Rightarrow$ row removed

AND/OR/NOT + precedence (parentheses save you)

```
-- Students in CSE or EEE with GPA >= 3
SELECT Name, Dept, GPA
FROM school.Student
WHERE (Dept = 'CSE' OR Dept = 'EEE')
      AND GPA >= 3.00;
```

Precedence:

- ▶ NOT (highest)
- ▶ AND
- ▶ OR (lowest)

Rule: if there is both AND and OR, use parentheses.

ORDER BY: sorting results

```
-- Sort by GPA (highest first)
SELECT Name, GPA
FROM school.Student
ORDER BY GPA DESC;

-- Sort by Dept, then GPA within each dept
SELECT Name, Dept, GPA
FROM school.Student
ORDER BY Dept ASC, GPA DESC;
```

Sorting is the last step: it does not change which rows you have, only their order.

TOP: limiting rows (SQL Server)

```
-- Top 3 students by GPA
SELECT TOP 3 Name, GPA
FROM school.Student
WHERE GPA IS NOT NULL
ORDER BY GPA DESC;

-- TOP WITH TIES (keeps all tied rows at the boundary)
SELECT TOP 2 WITH TIES Name, GPA
FROM school.Student
WHERE GPA IS NOT NULL
ORDER BY GPA DESC;
```

Without ORDER BY, TOP is not meaningful (the "first" rows are arbitrary).

Common SELECT mistakes (and the fixed version)

Wrong

```
-- 1) NULL with =  
WHERE GPA = NULL  
  
-- 2) Text without quotes  
WHERE Dept = CSE  
  
-- 3) Mixing AND/OR without ()  
WHERE Dept='CSE' OR Dept='EEE'  
    AND GPA >= 3.0
```

Correct

```
WHERE GPA IS NULL  
  
WHERE Dept = 'CSE'  
  
WHERE (Dept='CSE' OR Dept='EEE')  
    AND GPA >= 3.0
```

Debug habit: copy your WHERE into a SELECT first, confirm the rows, then refine.

SELECT cheat sheet (syntax + memory lines)

Template (SQL Server)

```
SELECT [TOP n] columns  
FROM schema.Table  
[WHERE condition]  
[ORDER BY col ASC/DESC]
```

Two orders to remember

- ▶ Written: SELECT FROM WHERE ORDER BY
- ▶ Logical: **F-W-S-O** = FROM → WHERE → SELECT → ORDER BY

Common WHERE operators

- ▶ Comparison: =, <>, <, <=, >, >=
- ▶ Range: BETWEEN a AND b (inclusive)
- ▶ Set: IN (...) / NOT IN (...)
- ▶ Pattern: LIKE 'M%' / LIKE '%ali%'
- ▶ NULL: IS NULL / IS NOT NULL

Precedence

- ▶ NOT > AND > OR (use parentheses)

Debugging checklist (SSMS)

- 1 Confirm database: `SELECT DB_NAME();`
- 2 Confirm table name: use `school.Student` (schema prefix).
- 3 Text values must be in quotes: `Dept = 'CSE '`.
- 4 NULL uses `IS NULL / IS NOT NULL` (never `= NULL`).
- 5 If you have AND and OR, add parentheses.
- 6 Add `ORDER BY` only at the end; TOP needs `ORDER BY`.
- 7 When stuck: simplify to `SELECT TOP 5 * FROM ...` then rebuild step-by-step.

Mini-quiz: predict the output (no typing)

Using our sample data, predict which **names** appear.

Q1

```
SELECT Name
FROM school.Student
WHERE GPA >= 3.5 OR GPA IS NULL;
```

Q2

```
SELECT Name
FROM school.Student
WHERE Dept = 'CSE' OR Dept = 'EEE'
      AND GPA >= 3.5;
```

Q3

```
SELECT Name
FROM school.Student
WHERE (Dept = 'CSE' OR Dept = 'EEE'
      AND GPA >= 3.5;
```

Discussion prompts:

- ▶ Why does Q2 include more rows than Q3?
- ▶ Which operator is evaluated first: AND or OR?

Mini-quiz solutions

Using the inserted sample data:

- ▶ **Q1** ($\text{GPA} \geq 3.5$ OR GPA IS NULL): **Asha Ali, Salma Noor, Mary K.**
- ▶ **Q2** ($\text{Dept} = \text{CSE}$ OR ($\text{Dept} = \text{EEE}$ AND $\text{GPA} \geq 3.5$)): **Asha Ali, Musa Omar, Mary K.**
- ▶ **Q3** (($\text{Dept} = \text{CSE}$ OR $\text{Dept} = \text{EEE}$) AND $\text{GPA} \geq 3.5$): **Asha Ali, Mary K.**

Key point: in Q2, AND is evaluated before OR, so **all CSE students** are included even if GPA is below 3.5.

Live quiz rules (do this in class)

- ▶ Work on your own machine (SSMS).
- ▶ You may discuss with your neighbor, but each student submits their own SQL.
- ▶ Run every query: if it errors, fix it.
- ▶ For each question: **write the SQL** and **show the result grid**.

Reminder: confirm DB first: `SELECT DB_NAME();`

Assessment (Part A): write these 6 queries

Use the dataset we created.

- 1 List all student names in alphabetical order.
- 2 Show Name and GPA for students with GPA at least 3.50.
- 3 List distinct departments in the Student table.
- 4 Show students whose name starts with letter M.
- 5 Show students in CSE or EEE with GPA **at least 3.00**. (Hint: parentheses.)
- 6 Show the top 2 (with ties) GPAs (ignore NULL GPAs).

Assessment (Part B): debugging challenge

This query is wrong. Fix it and run it.

```
-- BUGGY QUERY
SELECT Name, GPA
FROM school.Student
WHERE GPA = NULL OR Dept = CSE
ORDER BY GPA desc;
```

You must fix: NULL handling, quotes for text, and (optionally) formatting.

Solutions (Part A): reference answers

-- Q1) Names A->Z

```
SELECT Name
FROM school.Student
ORDER BY Name ASC;
```

-- Q2) GPA >= 3.50

```
SELECT Name, GPA
FROM school.Student
WHERE GPA >= 3.50;
```

-- Q3) Distinct departments

```
SELECT DISTINCT Dept
FROM school.Student;
```

Solutions (Part A): reference answers (continued)

-- Q4) Names starting with M

```
SELECT Name
FROM school.Student
WHERE Name LIKE 'M%';
```

-- Q5) (CSE or EEE) and GPA >= 3.00

```
SELECT Name, Dept, GPA
FROM school.Student
WHERE (Dept = 'CSE' OR Dept = 'EEE')
      AND GPA >= 3.00;
```

-- Q6) Top 2 GPAs with ties (ignore NULL)

```
SELECT TOP 2 WITH TIES Name, GPA
FROM school.Student
WHERE GPA IS NOT NULL
ORDER BY GPA DESC;
```

Solution (Part B): fixed debugging query

```
-- Fixed version
SELECT Name, GPA
FROM school.Student
WHERE GPA IS NULL OR Dept = 'CSE'
ORDER BY GPA DESC;
```

This works because: IS NULL handles NULL, and text values must be in quotes.

Key Takeaways

- ✓ Always write: `SELECT (columns) FROM (table) WHERE (filter) ORDER BY (sort)`.
- ✓ Logical evaluation: `FROM` $\rightarrow$ `WHERE` $\rightarrow$ `SELECT` $\rightarrow$ `ORDER BY`.
- ✓ `NULL` requires `IS NULL` / `IS NOT NULL`.
- ✓ Use parentheses when mixing `AND` and `OR`.
- ✓ `TOP` needs `ORDER BY` to be meaningful.

Next lecture: SQL DML — `INSERT`, `UPDATE`, `DELETE` (and why transactions matter).